Министерство образования Республики Беларусь
Учреждение образования
«Белорусский государственный университет информатики
и радиоэлектроники»

Кафедра программного обеспечения информационных технологий

Специальность «Программная инженерия»

Учебная дисциплина «Метрология, стандартизация и сертификация (в информационных технологиях)»

ОТЧЕТ

к практическому занятию №2а
«Метрики сложности потока управления программ на примере метрики Джилба»

Подготовили: Ковалевич К.А.

Заверач С.П.

Проверила: Болтак С.В.

Минск 2026

Код анализируемой программы (язык программирования Swift):

```
func fullGilbMetricsAnalysis() {
        let array = [1, 2, -3, 4, 0]
        var sum = 0

        let limit = array.isEmpty ? 0 : array.count

        for num in array {
            guard num != 0 else { continue }

            if num > 0 {
                var temp = num

                while temp > 0 {
                    switch temp {
                    case 1:
                        sum += 10
                    case 2:
                        sum += 20
                    default:
                        sum += temp
                    }
                    temp -= 1
                }
            } else if num < 0 {
                var k = 0

                repeat {
                    sum -= 1
                    k += 1
                } while k < abs(num)
            }
        }

        array.forEach { item in
            sum += 1
        }

        let code = 404
        var status = ""

        switch code {
        case 200:
            status = "OK"
        case 400:
            status = "Bad Request"
        case 403:
            status = "Forbidden"
        case 404:
            status = "Not Found"
        case 500:
            status = "Server Error"
```

```
        default:
            status = "Unknown Error"
        }

        let numbers = [3, -2, 4, 0]
        var result = 0

        numbers.forEach { num in
            let multiplier = num > 0 ? 2 : 1

            for i in 0..<abs(num) {
                if i % 2 == 0 {
                    var count = i

                    while count < 3 {
                        switch count {
                        case 0:
                            result += 1 * multiplier
                        case 1:
                            result += 2 * multiplier
                        case 2:
                            result += 3 * multiplier
                        default:
                            var k = 0

                            repeat {
                                result -= 1
                                k += 1
                            } while k < 2
                        }
                        count += 1
                    }
                }
            }
        }

        print("Sum: \(sum), Status: \(status), Result:
\(result)")
    }
```

Блок-схема алгоритма.

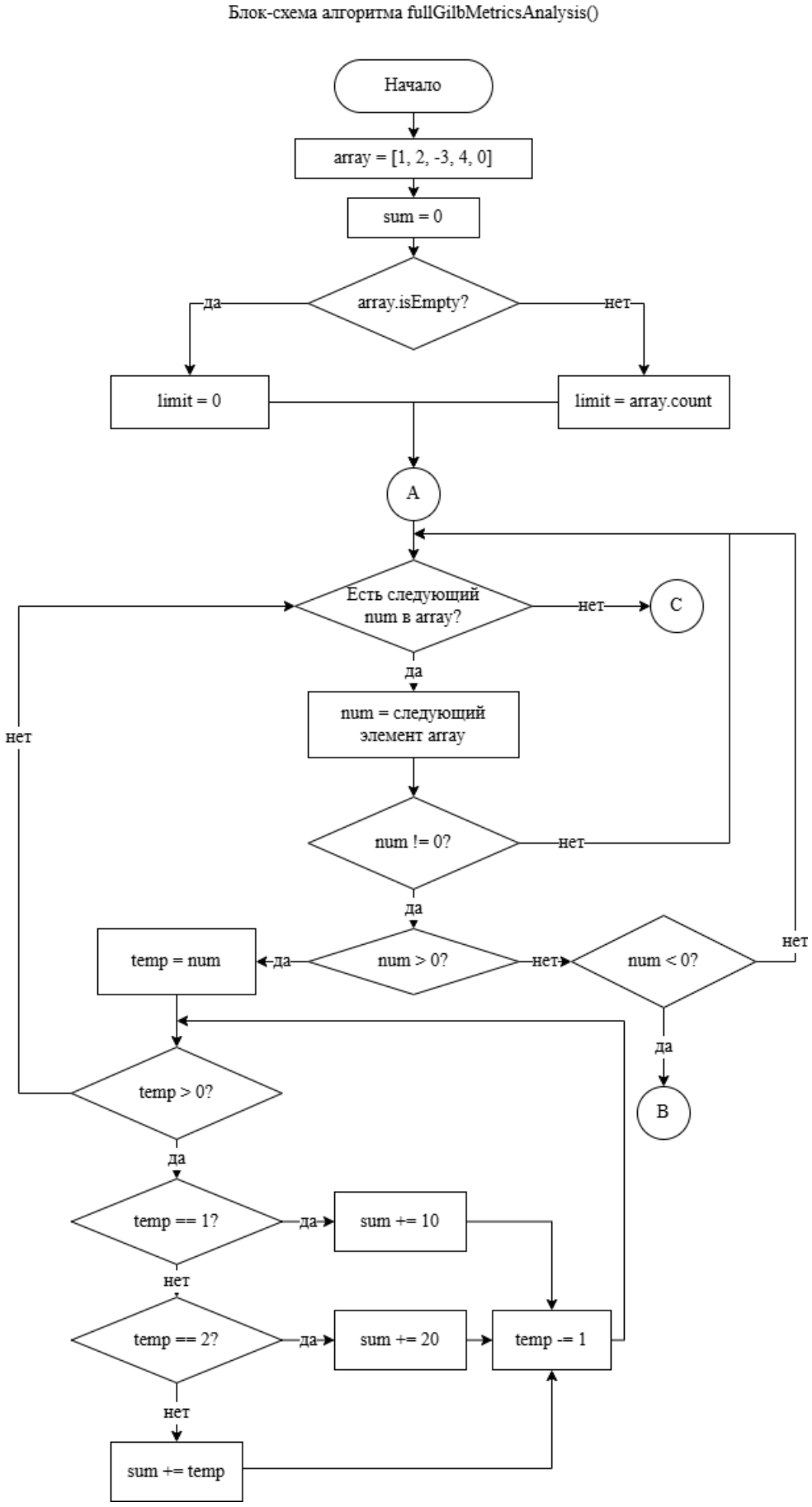


Рисунок 1 – Блок-схема алгоритма

Продолжение: ветка num < 0, forEach и switch code

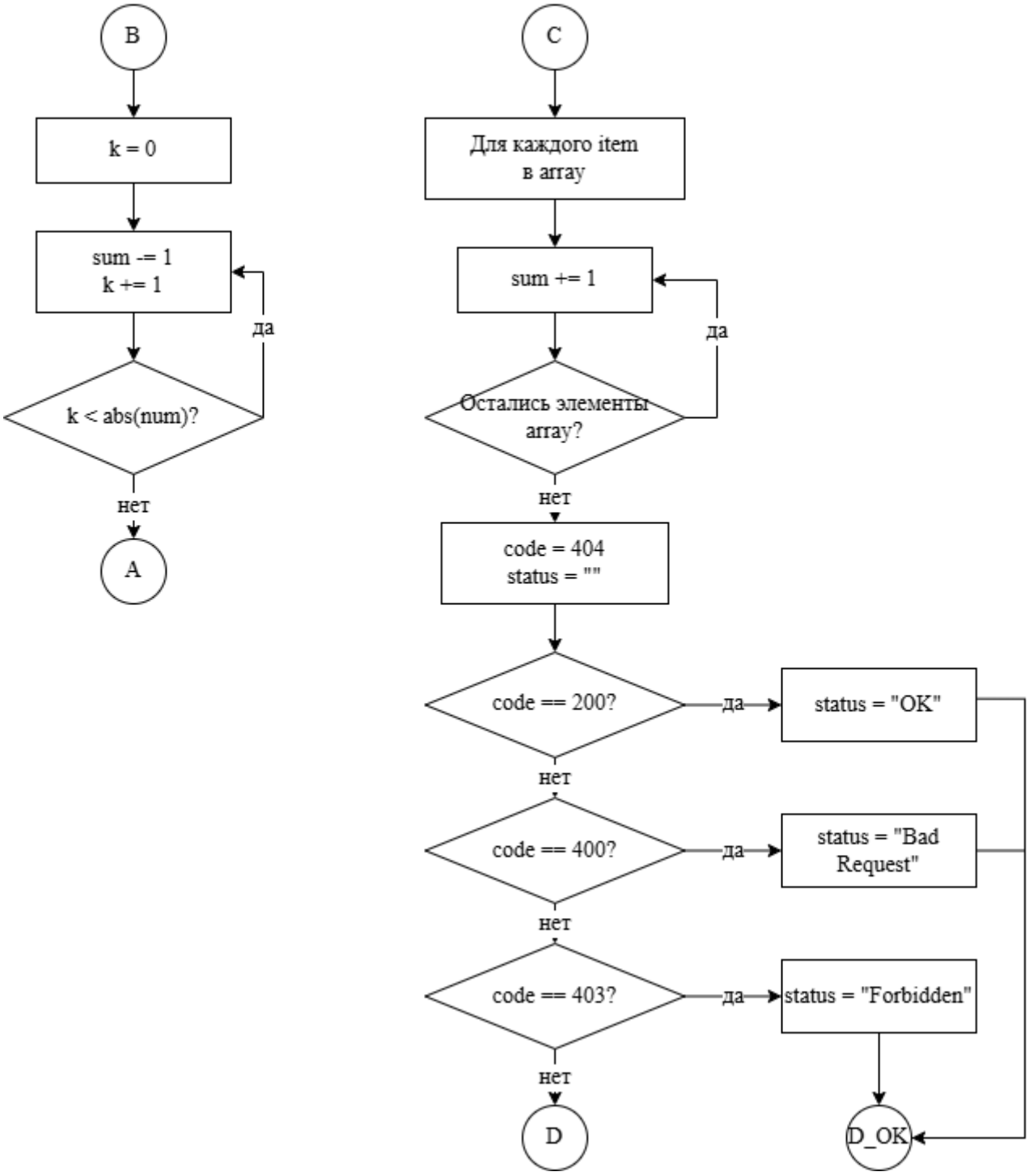


Рисунок 2 – Блок-схема алгоритма продолжение

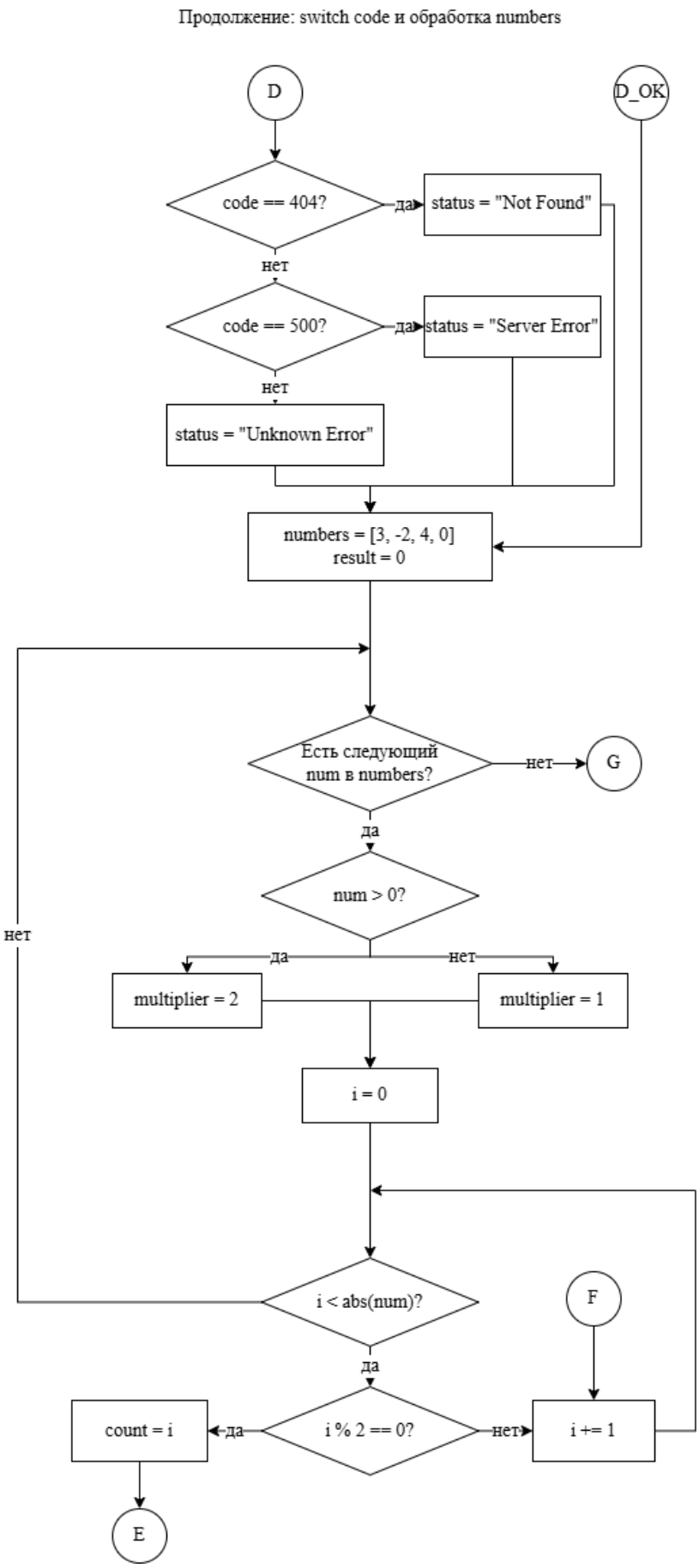


Рисунок 3 – Блок-схема алгоритма продолжение

Продолжение: внутренний switch, repeat-while и завершение

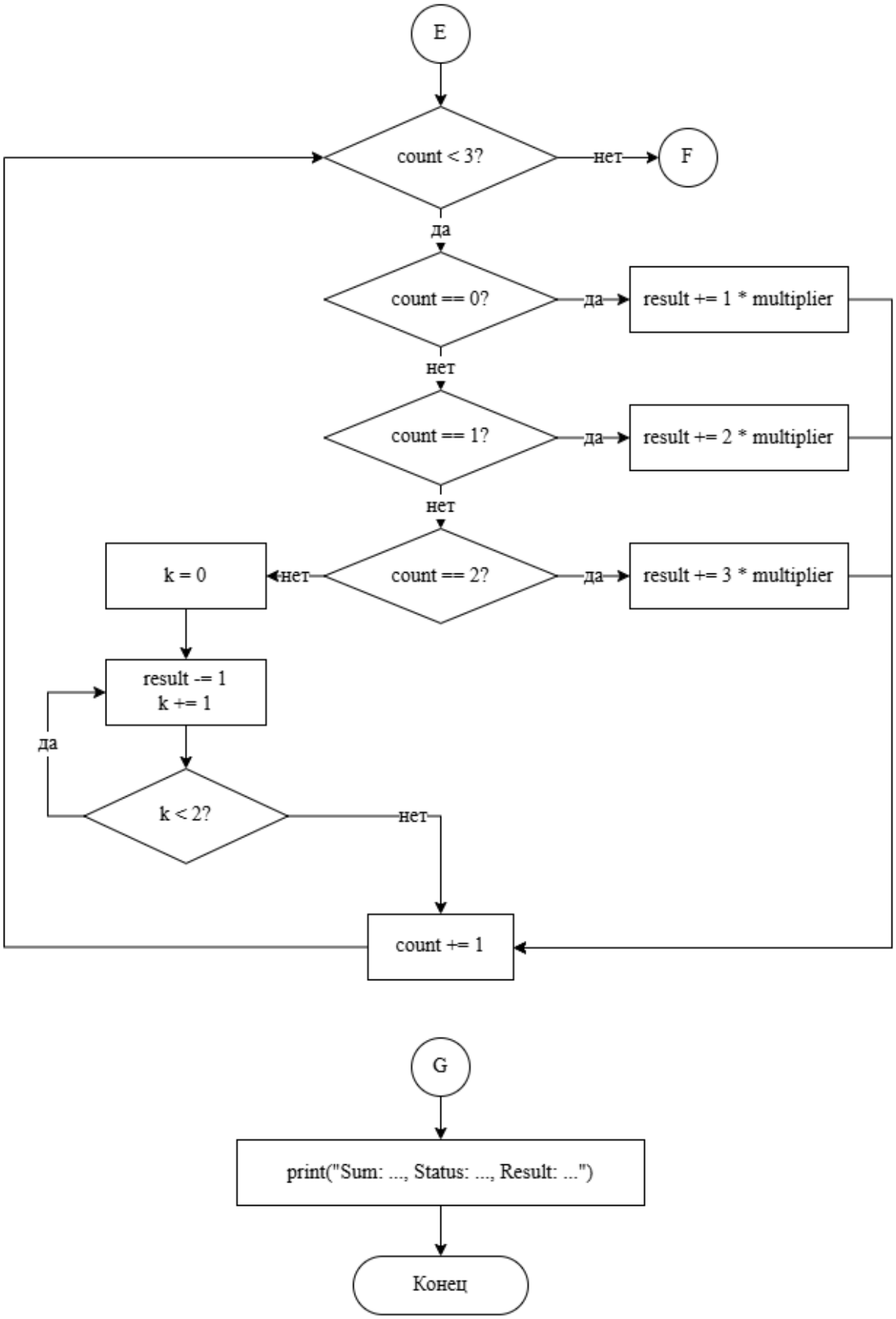


Рисунок 4 – Блок-схема алгоритма продолжение

Результат работы программы:



Рисунок 5 – Результат работы программы

Ручной подсчёт метрики Джилба:

Для расчёта операторы switch представлены в виде эквивалентных вложенных условных операторов. Объявление с инициализацией, присваивание, самостоятельный вызов функции и управляющая конструкция считаются отдельными операторами. Выражения внутри них отдельно не учитываются.

Для ручного подсчета используем эквивалентное развертывание структур:

```
func fullGilbMetricsAnalysis() {

    let array = [1, 2, -3, 4, 0]          // N1
    var sum = 0                           // N2

    let limit = array.isEmpty ? 0 : array.count
    // N3, CL1

    for num in array {
        // N4, CL2
```

```
    guard num != 0 else { continue }
    // N5, CL3
    // continue НЕ считаем

    if num > 0 {
        // N6, CL4

        var temp = num                  // N7

        while temp > 0 {
            // N8, CL5

            switch temp {

            case 1:
                // N9, CL6
                sum += 10               // N10

            case 2:
                // N11, CL7
                sum += 20               // N12

            default:
                sum += temp             // N13
            }

            temp -= 1                   // N14
        }

    } else if num < 0 {
        // N15, CL8

        var k = 0                       // N16

        repeat {
            // N17, CL9

            sum -= 1                    // N18
            k += 1                      // N19

        } while k < abs(num)
    }
}

array.forEach { item in
    // N20, CL10

    sum += 1                            // N21
}

let code = 404                          // N22
var status = ""                         // N23
```

```
switch code {

case 200:
    // N24, CL11
    status = "OK"                       // N25

case 400:
    // N26, CL12
    status = "Bad Request"              // N27

case 403:
    // N28, CL13
    status = "Forbidden"                // N29

case 404:
    // N30, CL14
    status = "Not Found"                // N31

case 500:
    // N32, CL15
    status = "Server Error"              // N33

default:
    status = "Unknown Error"             // N34
}

let numbers = [3, -2, 4, 0]             // N35
var result = 0                           // N36

numbers.forEach { num in
    // N37, CL16

    let multiplier = num > 0 ? 2 : 1
    // N38, CL17

    for i in 0..<abs(num) {
        // N39, CL18

        if i % 2 == 0 {
            // N40, CL19

            var count = i                // N41

            while count < 3 {
                // N42, CL20

                switch count {

                case 0:
                    // N43, CL21
                    result += 1 * multiplier     // N44

                case 1:
```

```
                        // N45, CL22
                        result += 2 * multiplier     // N46

                    case 2:
                        // N47, CL23
                        result += 3 * multiplier     // N48

                    default:
                        var k = 0                    // N49

                        repeat {
                            // N50, CL24

                            result -= 1              // N51
                            k += 1                   // N52

                        } while k < 2
                    }

                    count += 1                       // N53
                }
            }
        }
    }

    print("Sum: \(sum), Status: \(status), Result:
\(result)")
    //
}
```

1. Абсолютная сложность – CL

Считаем управляющие конструкции:

Таблица 1 – Подсчет управлябщих конструкций

№ оператора	**Конструкция**	**Вклад в CL**
3	Тернарный оператор в limit	1
4	Цикл for num in array	1
5	Условие guard (if num != 0)	1
6	Условие if num > 0	1
8	Цикл while temp > 0	1
9	Проверка temp == 1	1
11	Проверка temp == 2	1
15	Условие else if num < 0	1

17	Цикл repeat … while	1
20	Метод array.forEach	1
24	Проверка code == 200	1
26	Проверка code == 400	1
28	Проверка code == 403	1
30	Проверка code == 404	1
32	Проверка code == 500	1
37	Метод numbers.forEach	1
38	Тернарный оператор в multiplier	1
39	Цикл for i in 0..<abs(num)	1
40	Условие if i % 2 == 0	1
42	Цикл while count < 3	1
43	Проверка count == 0	1
45	Проверка count == 1	1
47	Проверка count == 2	1
50	Вложенный цикл repeat … while	1
	Итого	**24**

Обычный else не содержит новой проверки и отдельно не увеличивает CL.

2. Общее количество операторов – N

Таблица 2 – Подсчет операторов

Вид операторов	**Номера**	**Количество**
Объявления с инициализацией	1, 2, 7, 16, 22, 23, 35, 36, 41, 49	10
Управляющие конструкции	3, 4, 5, 6, 8, 9, 11, 15, 17, 20, 24, 26, 28, 30, 32, 37, 38, 39, 40, 42, 43, 45, 47, 50	24 (включая ветвления/проверки)

Изменения значений переменных	10, 12, 13, 14, 18, 19, 21, 25, 27, 29, 31, 33, 34, 44, 46, 48, 51, 52, 53	19
Итого		**53**

В автоматическом анализаторе зафиксировано значение 53.

3. Относительная сложность – cl

$$cl = \frac{CL}{N} = \frac{24}{53} \approx 0,4528$$

4. Максимальная вложенность – CLI
Внешняя управляющая конструкция имеет уровень 0.

Таблица 3 – Самая глубокая цепочка

Конструкция	**Уровень**
numbers.forEach	0
for i in 0..<abs(num)	1
if i % 2 == 0	2
while count < 3	3
Эквивалентный if-else от switch count	4
Вложенный цикл repeat … while в ветке default	5

Следовательно:
CLI=5

Таблица 4 – Итоговая таблица подсчетов

Метрика	**Значение**
Абсолютная сложность CL	**24**
Общее количество операторов N	**53**
Относительная сложность cl	**0,4528**
Максимальная вложенность CLI	**5**

Расчёт выполнен для алгоритма после представления операторов switch в виде вложенных условных операторов. Внешний уровень вложенности принят равным нулю.